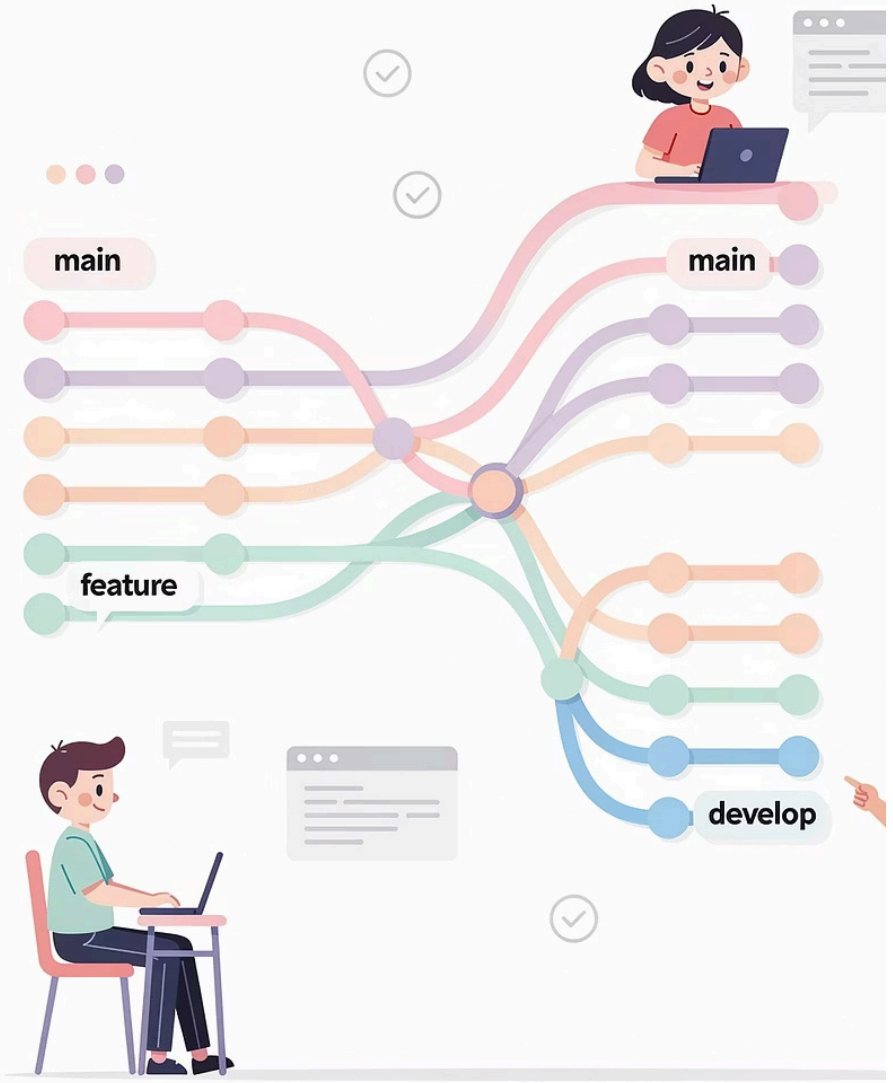
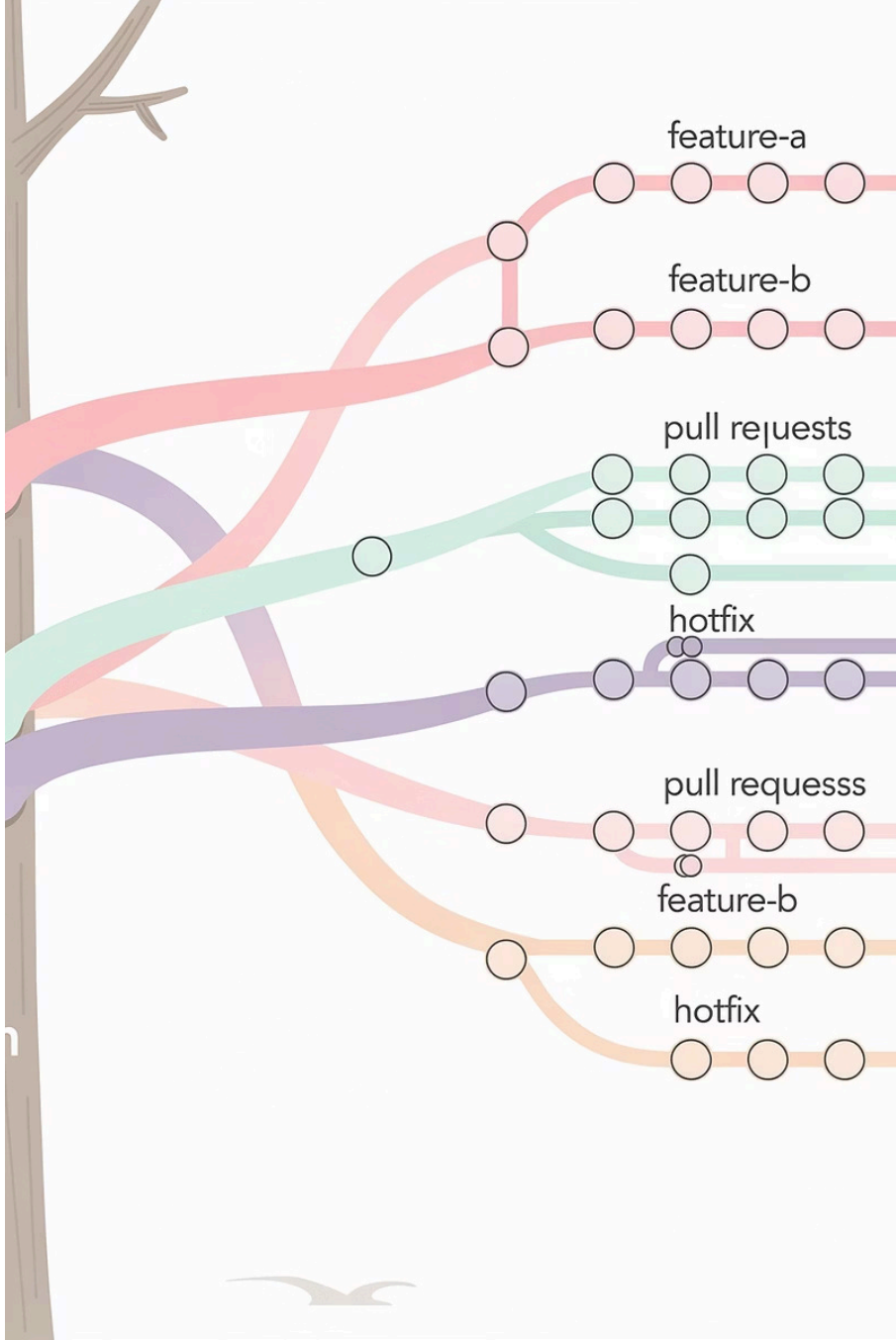


שיעור 3 - Git

Branches, Rebase & Merge

Branches

ענפים (branches) בGitHub הם דרך להפריד בין גרסאות שונות של קוד. הם מאפשרים למפתחים לעבוד על שינויים בקוד מבלי להשפיע על גרסה יציבה של הקוד.



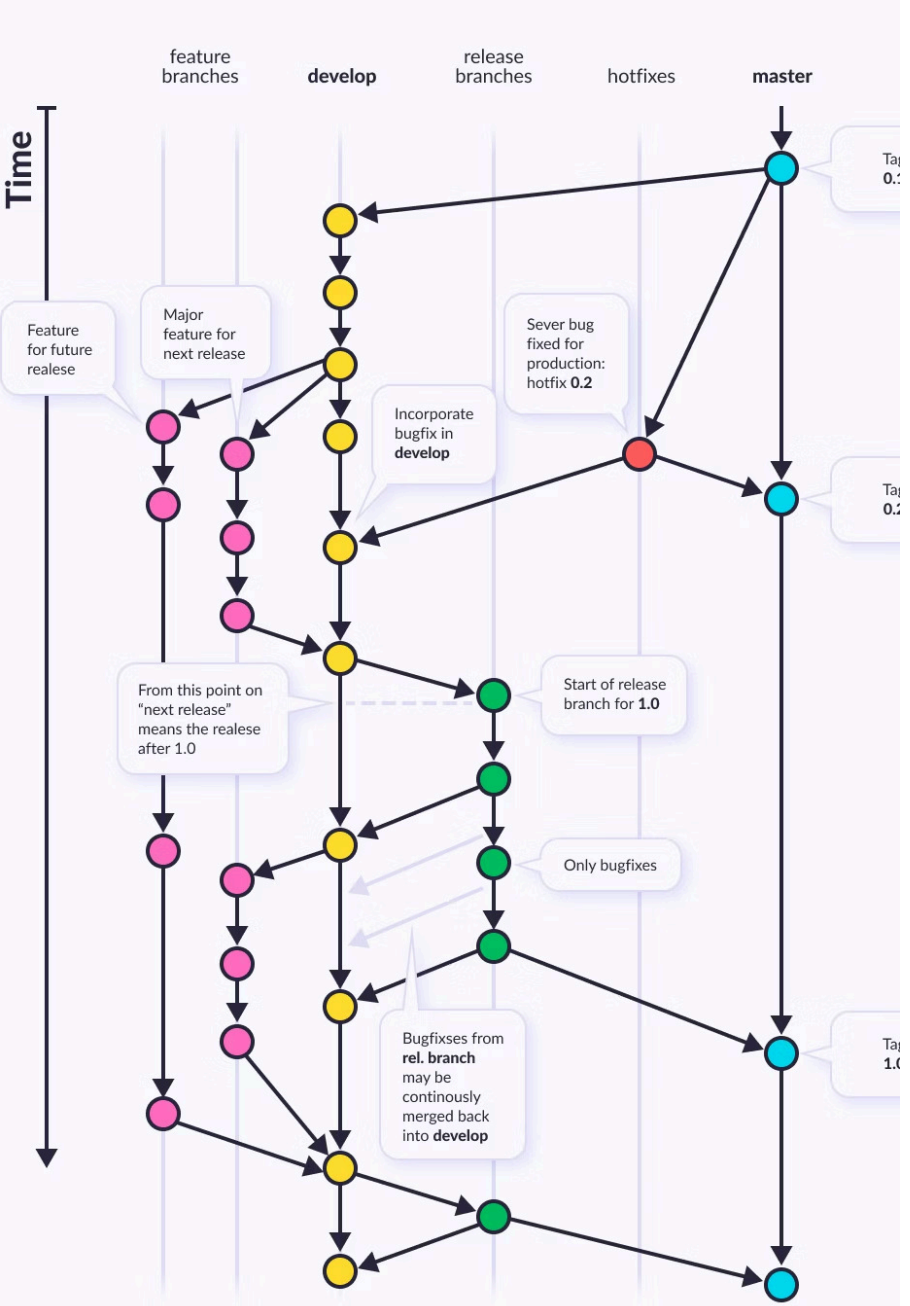
מה הם ענפים?

המטרה

הם מאפשרים למפתחים לעבוד על שינויים בקוד **מבלי להשפיע על גרסה יציבה של הקוד**.

הגדרה

ענפים (branches) בGitHub הם דרך להפריד בין גרסאות שונות של קוד.



יתרונות הענפים בGitHub

🛡️ בטיחות

ענפים מאפשרים למפתחים לעבוד על שינויים חדשים מבלי להשפיע על הקוד הפעיל. אם משהו משתבש, ניתן פשוט להחזיר את הענף למצבו הקודם.

⚡ יעילות

ענפים מאפשרים למפתחים לעבוד על משימות שונות במקביל. ניתן לפתוח ענף חדש לכל משימה, כך שלא צריך לדאוג שהשינויים של משימה אחת ישפיעו על משימה אחרת.

🔄 גמישות

ענפים מאפשרים למפתחים לנסות דברים חדשים מבלי להשפיע על הקוד הפעיל. ניתן לפתוח ענף חדש כדי לנסות פיתוח חדש, ולאחר מכן למזג את השינויים לענף הראשי אם הם מצליחים.

סוגי ענפים בGitHub

1

ענף ראשי: `main`

זהו הענף העיקרי של המאגר. הוא מכיל את **הגרסה היציבה** של הקוד.

2

ענף פיתוח: `development`

זהו ענף שבו מתבצעים שינויים בקוד. הוא משמש בדרך כלל כ**בסיס לענפים אחרים**.

3

ענף פיצ'ר: `feature`

זהו ענף שבו **מתווספות תכונות חדשות** לקוד.

4

ענף תיקון באג: `bugfix`

זהו ענף שבו **מתוקנים באגים** בקוד.

כמובן ששמות הענפים הם לבחירה אישית — feature יהיה כשם הfeature וכן הלאה...

דוגמאות לשימוש בענפים

1 פיתוח מערכת חדשה

חברה פיתחה מערכת חדשה. החברה פתחה ענף פיתוח חדש כדי לעבוד על המערכת. לאחר שהמערכת הייתה מוכנה, החברה מיזגה את הענף לענף הראשי.

2 תיקון באג בתוכנה

מפתח מצא באג בתוכנה. המפתח פתח ענף תיקון באגים חדש כדי לתקן את הבאג. לאחר שהבאג תוקן, המפתח מיזג את הענף לענף הראשי.

3 בדיקת שינויים חדשים

צוות פיתוח בודק שינויים חדשים בקוד. הצוות פתח ענף חדש כדי לבדוק את השינויים. לאחר שהשינויים נבדקו, הצוות מיזג את הענף לענף הראשי.

ענפים הם כלי חשוב לניהול גרסאות קוד. הם מאפשרים למפתחים לעבוד על שינויים בקוד מבלי להשפיע על הקוד הפעיל. הם גם תורמים לבטיחות, יעילות וגמישות של תהליך הפיתוח.

בואו נתחיל! יצירת ענף חדש

על מנת ליצור ענף חדש, נשתמש בפקודה הבאה:

```
git branch newbranch
```

על מנת לקבל את רשימת הענפים שיש ברשותנו:

```
git branch
```

כפי שניתן לראות בתמונה, ענף אחד צבוע בירוק ובעל כוכבית. המשמעות היא שהענף הזה הוא הענף שאנו עובדים עליו כרגע.

מעבר בין ענפים

איך נעבור לענף אחר?

```
git checkout newbranch
```

ניתן לראות כי גיט בישרה לנו כי ביצענו החלפה לענף בשם `newbranch`.

ניתן ליצור ענף חדש ולעבור עליו באמצעות פקודה אחת בלבד:

```
git checkout -b version2
```

אם נפתח ענף חדש פיצ'ר בשם `feature` ברפו המקומי בשביל לקבוע קשר וליצור ענף ברפו המרוחק – `origin/feature` נצטרך להריץ את הפקודה הבאה (גיט ינחה אותנו להריץ אותה):

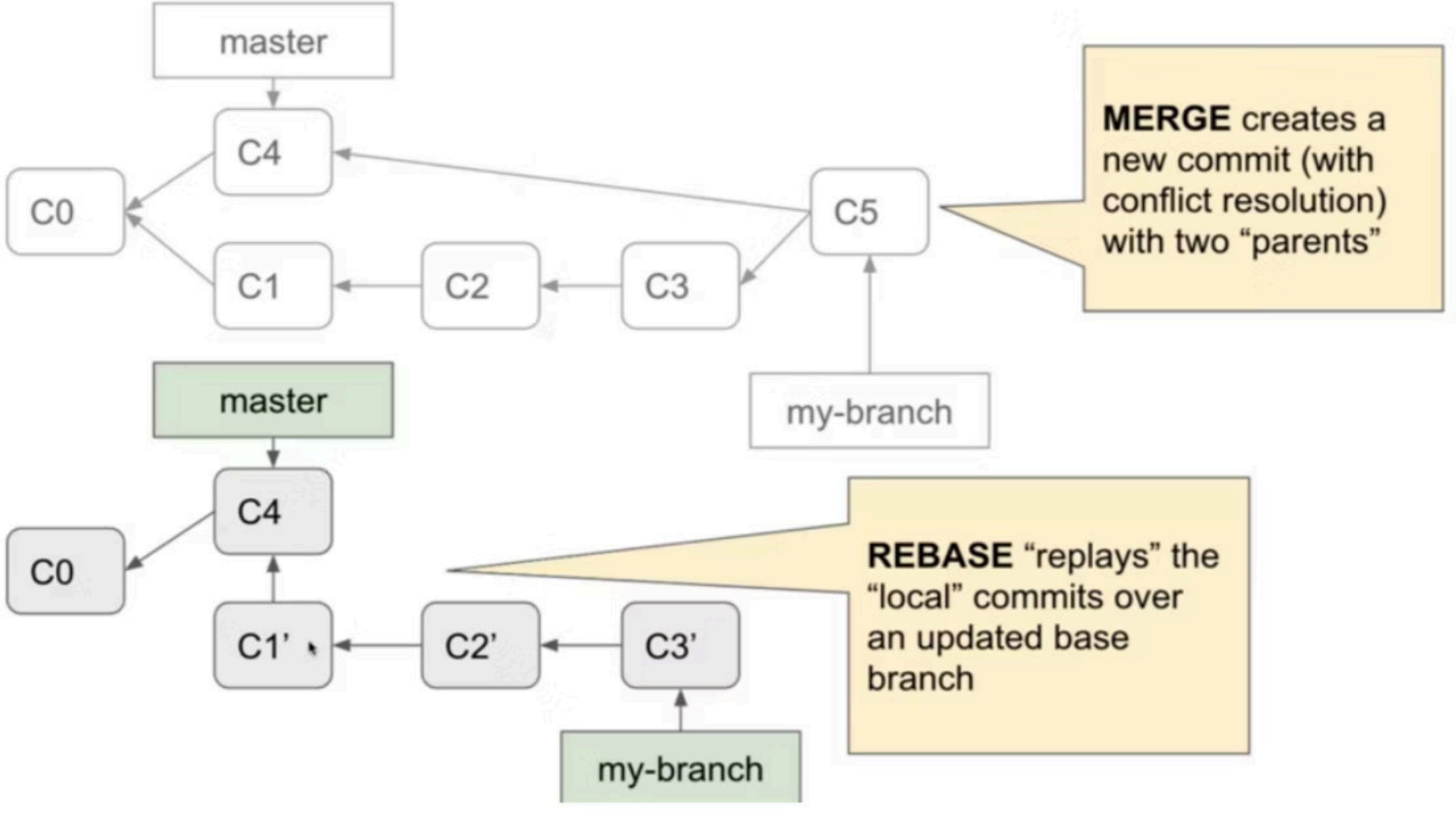
```
git push --set-upstream origin feature
```

וכך עכשיו נוכל להשתמש בפקודות `pull` ו-`push` לעדכן את הענף המרוחק.

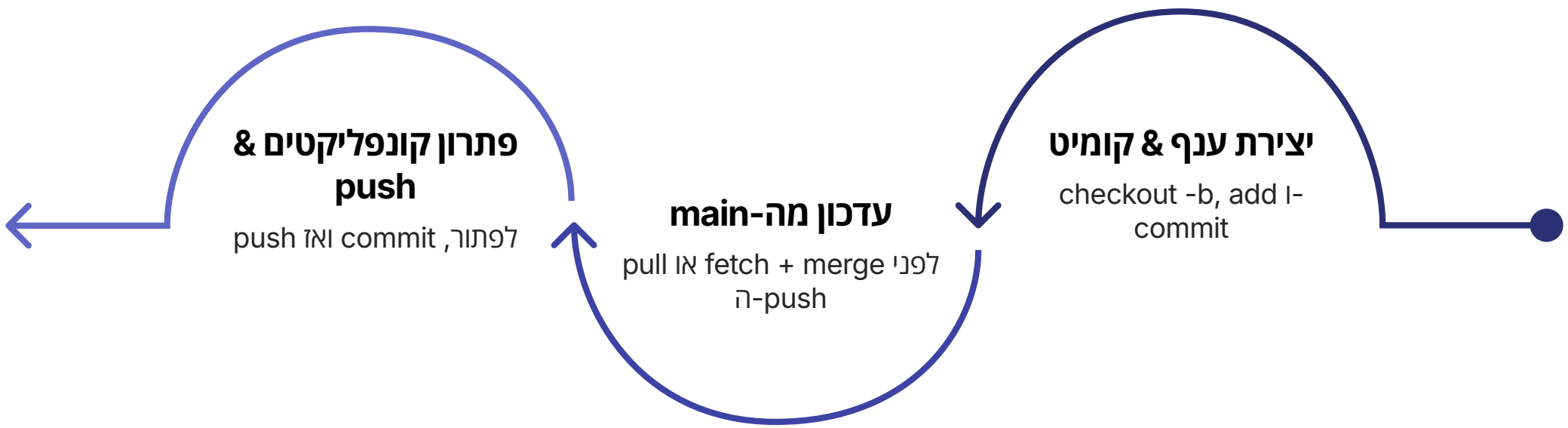
מיזוג ענפים — Merge vs Rebase

גם **Merge** וגם **Rebase** הן דרכים לשלב שינויים מענף אחד לענף אחר, אבל הן עובדות בצורה שונה ומשפיעות אחרת על היסטוריית הפרויקט.

נושא	Merge	Rebase
היסטוריה	משלב היסטוריות ע"י יצירת Merge Commit חדש, ושומר על מבנה הענפים	מזיז את הקומיטים שלך מעל ענף אחר ויוצר היסטוריה ליניארית
גרף קומיטים	מבנה מסועף נשמר	נראה כמו קו ישר
טיפול בקונפליקטים	פותרים קונפליקטים פעם אחת בזמן ה-merge	ייתכנו קונפליקטים בכל קומיט שמורץ מחדש
מתי להשתמש	כשחשוב לשמור את ההקשר וההיסטוריה המלאה	כשרוצים היסטוריה נקייה וליניארית


master
C4
C0
C1
C2
C3
C5
MERGE creates a new commit (with conflict resolution) with two “parents”
master
my-branch
C4
C0
C1’
C2’
C3’
REBASE “replays” the “local” commits over an updated base branch
my-branch

איך עובדים עם Merge



להלן הפקודות המלאות לעבודה עם Merge:

```
git checkout -b feature-branch   # יצירת ענף חדש ומעבר אליו
git add .
git commit -m "My feature work"  # עבודה וקומיט
git pull origin main             # עדכון הענף המקומי מה-main
```

אם יש קונפליקטים

- ה-IDE (למשל IntelliJ) יסמן אותם
- פותרים את הקונפליקטים

```
git add .
git commit # מסיים את ה-merge
git push origin feature-branch
```

אם אין קונפליקטים

```
git push origin main
```

איך עובדים עם Rebase

```
git checkout -b feature-branch   # יצירת ענף חדש ומעבר אליו
git add .
git commit -m "My feature work"  # עבודה וקומיט
```

💡 בזמן העבודה מומלץ להשתמש ב: `git commit --amend` כדי לשמור על היסטוריה נקייה ולהקל על ה-rebase.

לאחר מכן:

```
git pull --rebase origin main   # מבצע rebase של הענף שלך על גבי ה-main המעודכן
```

אם יש קונפליקטים

1. פותרים אותם ב-IDE

```
git status       # לבדוק מה עוד צריך staging
git add .        # אם צריך, להוסיף קבצים שנפתרו
git rebase --continue
```

אם אין קונפליקטים

סיימת 🎉

🔁 אם יש כמה קומיטים בענף, ייתכן שתצטרכי לפתור קונפליקטים עבור כל קומיט בנפרד.

סיום ה-Rebase ו-Push

בסיום ה-rebase:

```
git push origin feature-branch
```

⚠️ אם הענף כבר נדחף קודם ל-remote:
`git push -f origin feature-branch`

Rebase	Merge
יוצר היסטוריה נקייה וישרה, אבל משנה את ההיסטוריה של הקומיטים.	שומר על היסטוריה מלאה עם מבנה ענפים ברור.

Squash Commits לפני Rebase

אם בענף שלך יש הרבה קומיטים קטנים, זמניים או לא מסודרים, מומלץ לבצע **Squash** ולאחד אותם לקומיט אחד ברור לפני שמבצעים Rebase.



היסטוריה נקייה

שומר על היסטוריה נקייה וברורה



פחות קונפליקטים

מקטין סיכוי לקונפליקטים מיותרים



פשוט יותר Rebase

הופך את ה-rebase לפשוט יותר

איך עושים Squash

```
git rebase -i HEAD~N
```

- מחליפים את `N` במספר הקומיטים האחרונים שברצונך לאחד.

בתפריט ה-Interactive Rebase:

→ משנים את המילה `pick` ל-`squash` (או `s`) עבור כל קומיט שרוצים למזג לתוך הקומיט שמעליו.

→ שומרים את הקובץ.

→ כותבים הודעת commit נקייה וברורה שמסכמת את כל השינויים.

עכשיו ההיסטוריה של הענף שלך נקייה, מסודרת ומוכנה ל-Rebase.

טיפ — לרוב נעדיף להשתמש במהלך העבודה ב `commit --amend` בשביל לחסוך squash



המלצות עבודה עם Git — Best Practices

השתמשי ב־`git commit --amend` כשאפשר

כדי להימנע מקומיטים מיותרים ולשמור על היסטוריה נקייה.

העדיפי Rebase

כדי לשמור על היסטוריה ליניארית, ברורה וקלה להבנה.

בצעי Rebase לעיתים קרובות

מומלץ לפחות פעם ביום — זה מפחית סיכוי לקונפליקטים ושומר אותך מעודכנת.

Pull Request (PR)

מה זה PR?

Pull Request (PR) הוא בקשה למזג שינויים מענף אחד לענף אחר (לרוב מ־feature ל־main). זה השלב שבו הקוד עובר בדיקה לפני שהוא נכנס לענף המרכזי.

מה המטרה של PR?

- לאפשר Code Review
- להריץ בדיקות אוטומטיות (CI)
- לוודא שהשינויים לא שוברים את המערכת
- לתעד מה השתנה ולמה

שלבי יצירת Pull Request

עוברים לממשק באתר **github.com**

01

בענף החדש, נלחץ על הכפתור הירוק **create a pull request**

02

בדף שיפתח נוסיף **כותרת והערות** בשביל להסביר את השינויים.

03

ננהל **דיון עם השותפים** לפרויקט.

04

ובסוף ניתן למזג: `merge pull request → confirm`

05

ועכשיו הענף המשני **ממוזג ל-main**.

06

ועכשיו כשעוברים לענף ה-main אפשר לראות את **השינויים**.

🔍 CR - Code Review

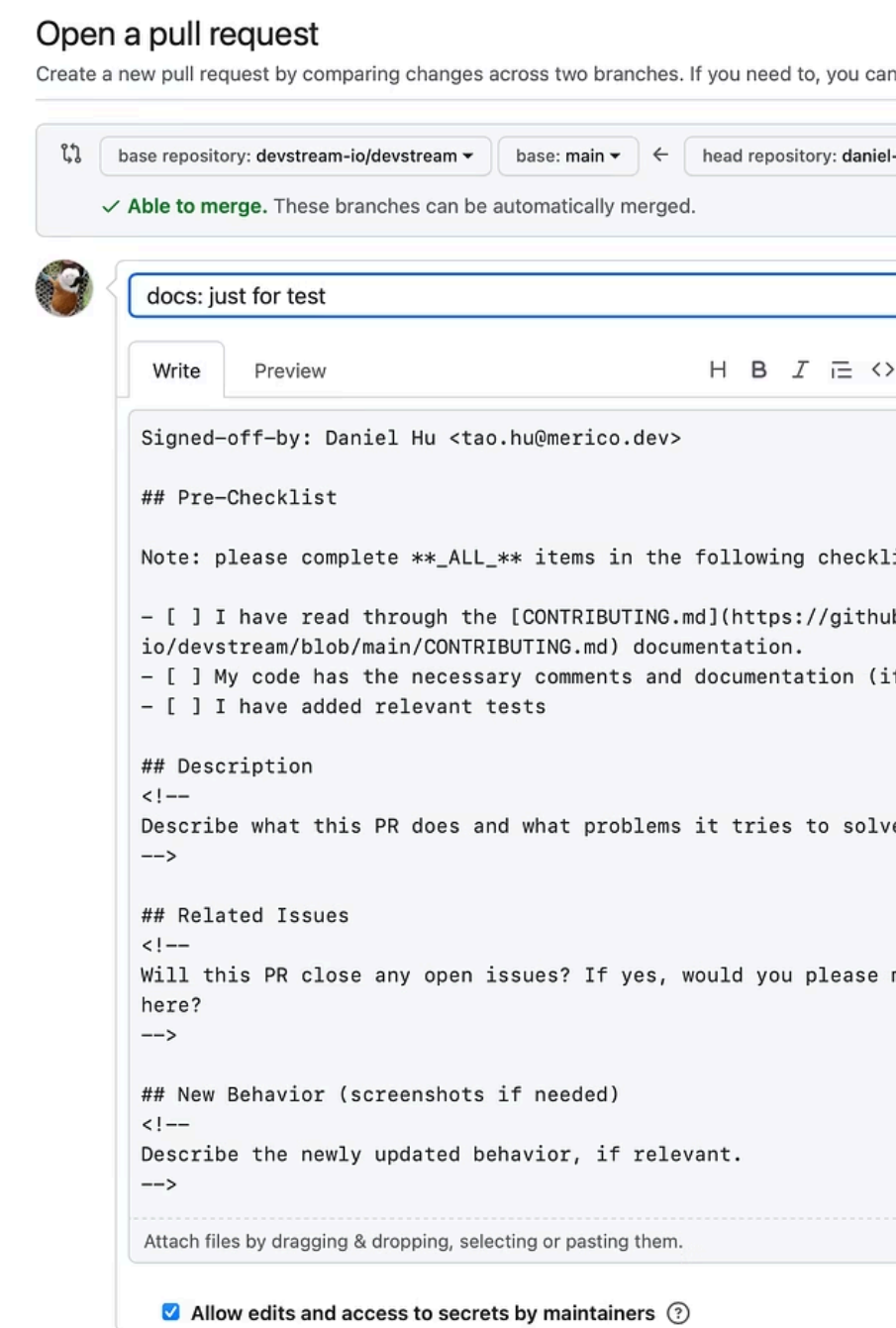
Code Review (CR) היא תהליך שבו מפתחים אחרים בצוות עוברים על הקוד שכתבת לפני שהוא נכנס ל־main (או לענף מרכזי אחר). המטרה היא לשפר את איכות הקוד, למנוע באגים ולשמור על סטנדרט אחיד בצוות.

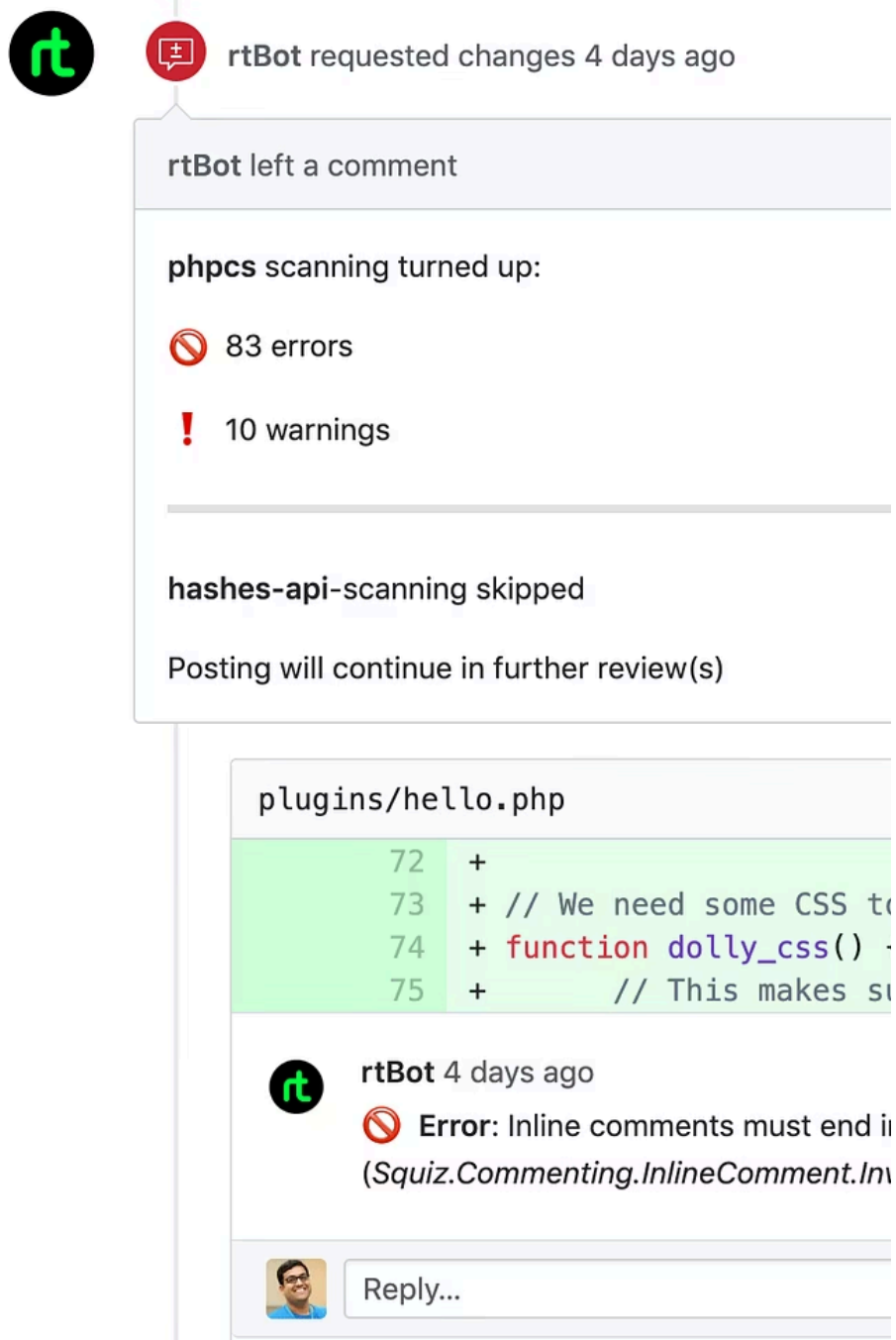
למה עושים Code Review?

- לאתר באגים ובעיות לוגיות לפני פרודקשן
- לשמור על סטנדרטים אחידים של קוד
- לשתף ידע בין חברי הצוות
- לשפר חשיבה ארכיטקטונית ותכנון
- לוודא שאין בעיות אבטחה או ביצועים

איך עושים CR נכון - ככותבת הקוד

- לפתוח PR ברור עם **תיאור מסודר**
- להסביר **מה השתנה ולמה**
- לשמור על **PR קטן וממוקד**
- להגיב להערות בצורה **עניינית**





איך עושים CR נכון - כבודקת הקוד

- להעיר בצורה **מכבדת ומקצועית**
- להתמקד **בקוד, לא באדם**
- להציע **שיפורים** ולא רק להצביע על בעיות
- להבחין בין **חובה לשיפור נחמד** (nit)

סיכום - Branches בGitHub

Branches

כלי חיוני להפרדת גרסאות קוד — בטיחות, יעילות וגמישות.

Merge vs Rebase

יוצר היסטוריה נקייה וליניארית; Rebase שומר היסטוריה מלאה Merge.

Pull Request

שלב הבדיקה לפני מיזוג לענף הראשי — Code Review, CI ותיעוד.

Best Practices

השתמשי ב-`commit --amend`, העדיפי Rebase, ובצעי אותו לעיתים קרובות.